

Ethan Patterson
Nico Cabrera

Algo HW3 Q4

a.) The algorithm uses a dynamic programming foundation to find the solution to every “sub problem” and use those to build up to the solution for the full hopscotch array. We track 4 values: sum of the square 3 places behind the current place, sum of the square 2 places behind the current place, sum of the square 1 square behind the current place(the “skip square”), and sum of the current square. At the start, these are all initialized to 0. For any given square, we can reach it in two ways: jump from 3 slots back, or jump from 2 slots back. The biggest sum of that square is always going to be a matter of what's bigger- sum of the 3 slots back slot or 2 slots back. When we figure out what one's bigger(ties produce the same result, doesn't matter, pick one), add the current square's value to that sum, and you have the highest sum that could reach this square(our current sum). Then before moving to the next one, the current square becomes the “skip square”, the “skip square” becomes 2 back, 2 back becomes 3 back, and 3 back is now irrelevant in future operations. Upon reaching the end of the array, there are two options: either the current sum or the skip square. Take the bigger one and that's your solution. There is a single edge case- the second square($i=1$ for zero based indexing) is unreachable in any situation. The score for this square is hardcoded to be 0, so it will always be beaten by any other possible sum, which will serve to remove it from processing.

b.) Pseudocode:

hopscotch(arr):

```
    3_back = 0
    2_back = 0
    Skip = 0
    Current = 0
    For i=0;i<arr.len;i++:
        Current = max(2_back, 3_back) + arr[i]
        3_back = 2_back
        2_back = skip
        Skip = current
    Return max(current, skip)
```

c.) The code iterates through all elements of the array from 0 to n. During each iteration, it checks if the sum of the current index is bigger when added to the value two indices back, or three. Whichever sum is larger, is the current sum, and so on. After the loop is complete, this will have calculated the largest possible sum at each element given the sums of the previous calculation. This atomic operation ensures that only the largest value is being taken into account for each element of the array, and added to the current element. Thus, when the maximum value is derived later, this maximum value will be the largest possible hopscotch combination.

d.) This algorithm runs in $O(n)$ time.

e.) The algorithm is $O(n)$ as it iterates through each element of an array of size n . At each step, it performs a constant number of operations (c). Then, it loops through all elements (n), again. Thus, it has a run time of $2cn$, or dropping lower order terms $O(n)$.